iOS Memory Management - Part I

Q) What is memory management in iOS?

A) Memory Management in iOS is the process of allocating and deallocating objects to prevent memory leaks and optimize app performance.

Q) How is memory managed in iOS?

A) iOS uses Automatic Reference Counting (ARC) to handle memory management automatically.

Q) What is ARC and how does it work?

A) ARC (Automatic Reference Counting) tracks object references. When the reference count reaches zero, the object is deallocated. ARC prevents memory leaks but can cause retain cycles if not managed properly.

Q) What is a retain cycle?

A) A retain cycle occurs when two or more objects hold strong references to each other, preventing deallocation and leading to memory leaks.

Q) How can retain cycles be avoided?

A) Retain cycles can be broken using weak or unowned references.

Q) What is a weak reference?

A) A weak reference does not increase an object's reference count, allowing it to be deallocated when no strong references remain.

Q) What is a strong reference?

A) A strong reference increases an object's reference count, ensuring it stays in memory as long as the reference exists.

Q) What is an unowned reference?

A) An unowned reference does not increase an object's reference count and does not become nil automatically. It is used when both objects have the same lifetime.

Q) Difference between weak and unowned?

- Weak: Can become nil, used for delegates or when an object may be deallocated.
- Unowned: Cannot be nil, used when both objects have the same lifetime.

Q) How does ARC handle closures in Swift?

A) Closures capture references, which may cause retain cycles if self is captured strongly. Using [weak self] or [unowned self] prevents this.

Example:

```
myClosure = { [weak self] in
    self?.doSomething()
}
```

Q) How can we identify memory leaks in iOS apps?

A) Memory leaks can be detected using:

- Xcode Instruments (Leaks Tool)
- Xcode Memory Graph Debugger
- Code reviews & static analysis